

1. Dlaczego dokumentacja API jest ważna?**

Wyobraź sobie, że budujesz skomplikowaną maszynę, ale nie tworzysz do niej żadnej instrukcji obsługi. Tylko Ty wiesz, jak jej używać. Każda nowa osoba, która będzie chciała z niej skorzystać, będzie musiała zgadywać albo ciągle pytać Cię o pomoc. Podobnie jest z API.

Info

API (Interfejs Programowania Aplikacji) to "kontrakt" lub umowa między serwerem (backendem) a klientem (np. aplikacją webową w przeglądarce lub aplikacją mobilną). Ta umowa precyzyjnie określa, jak obie strony mają się ze sobą komunikować: jakie dane można wysłać, w jakim formacie, i jakiej odpowiedzi można się spodziewać.

Dobra dokumentacja API jest jak czytelna instrukcja obsługi. Dzięki niej:

- **Frontend deweloperzy** wiedzą, jakich endpointów używać, jakie dane wysłać i co otrzymają w odpowiedzi, co znacznie przyspiesza ich pracę.
- **Nowi członkowie zespołu** mogą szybko zrozumieć, jak działa backend i zacząć pisać kod, który z niego korzysta.
- **Użytkownicy zewnętrzni** (jeśli Twoje API jest publiczne) mogą łatwo zintegrować swoje aplikacje z Twoim systemem.
- **Ty sam** za kilka miesięcy będziesz sobie wdzięczny, wracając do projektu i nie musząc analizować całego kodu, by przypomnieć sobie, jak działa konkretny endpoint.

```
graph TD
    subgraph Klient
        A[Aplikacja Frontend]
        B[Aplikacja Mobilna]
        C[Inny Serwer]
    end

    subgraph Serwer
        E[Aplikacja Backend w DRF]
    end

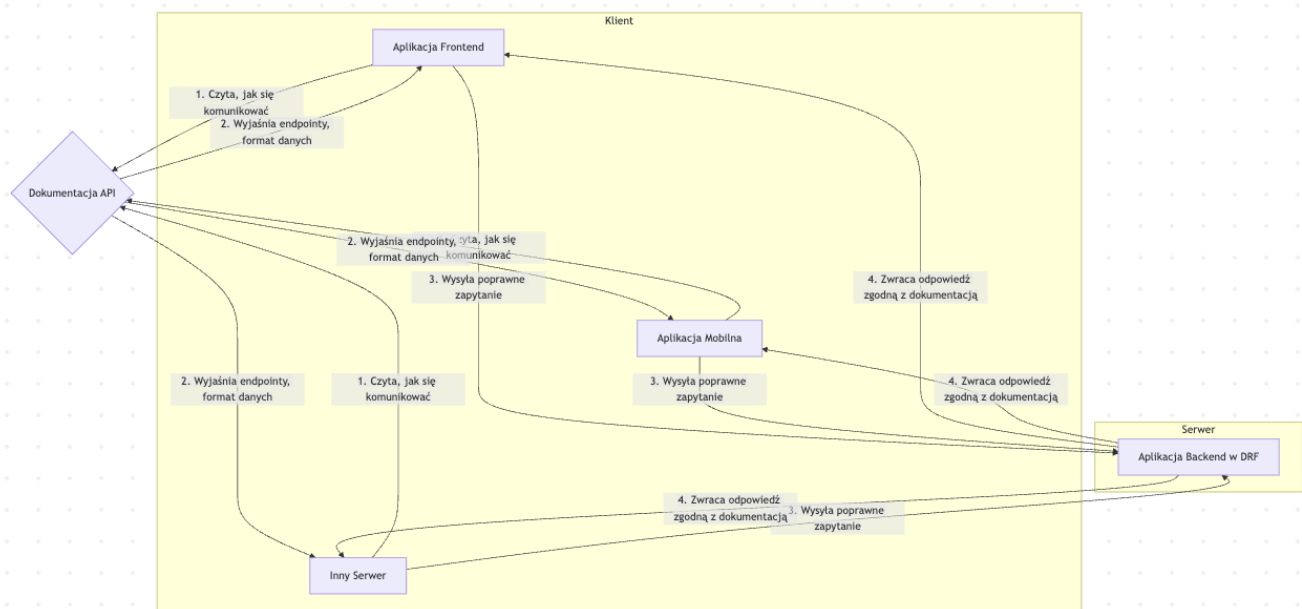
    D{Dokumentacja API}

    A -->|1. Czyta, jak się komunikować| D
    B -->|1. Czyta, jak się komunikować| D
    C -->|1. Czyta, jak się komunikować| D
```

D -->|2. Wyjaśnia endpointy, format danych| A
D -->|2. Wyjaśnia endpointy, format danych| B
D -->|2. Wyjaśnia endpointy, format danych| C

A -->|3. Wysyła poprawne zapytanie| E
B -->|3. Wysyła poprawne zapytanie| E
C -->|3. Wysyła poprawne zapytanie| E

E -->|4. Zwraca odpowiedź zgodną z dokumentacją| A
E -->|4. Zwraca odpowiedź zgodną z dokumentacją| B
E -->|4. Zwraca odpowiedź zgodną z dokumentacją| C



2. OpenAPI i Swagger – Co to jest?

Kiedy mówimy o dokumentacji API, prawie zawsze pojawiają się te dwa terminy. Ważne jest, aby zrozumieć różnicę między nimi.

Definition

OpenAPI Specification to standard (formalny opis, specyfikacja), który definiuje, jak opisywać API REST-owe. Jest to plik w formacie JSON lub YAML, który w ustrukturyzowany sposób zawiera informacje o wszystkich endpointach, metodach HTTP, parametrach, formatach danych i możliwych odpowiedziach. To jak "schemat" lub "plan" API.

Definition

Swagger to zestaw narzędzi (open-source i komercyjnych) zbudowanych wokół specyfikacji OpenAPI. Najpopularniejsze z nich to:

- **Swagger UI:** Generuje interaktywną, wizualną dokumentację API w przeglądarce na podstawie pliku OpenAPI. Umożliwia nawet testowanie endpointów bezpośrednio z poziomu dokumentacji.
- **Swagger Editor:** Edytor do tworzenia i walidacji plików specyfikacji OpenAPI.
- **Swagger Codegen:** Narzędzie, które potrafi wygenerować kod klienta (np. w Pythonie, JavaScriptcie) lub szkielet serwera na podstawie specyfikacji.

Podsumowując, OpenAPI to standard, a Swagger to narzędzia, które ten standard wykorzystują.

```
graph LR
    A["Specyfikacja OpenAPI<br>(plik .yaml/.json)"] -- jest czytana przez --> B("Narzędzia Swagger")
    B --> C["Swagger UI<br>(Interaktywna dokumentacja)"]
    B --> D["Swagger Editor<br>(Edytor specyfikacji)"]
    B --> E["Swagger Codegen<br>(Generator kodu)"]
```

"Screenshot 2025-10-08 at 14.14.23.png" could not be found.

3. Integracja Swagger UI z Django Rest Framework

Ręczne pisanie specyfikacji OpenAPI byłoby bardzo czasochłonne. Na szczęście istnieją biblioteki, które potrafią przeanalizować nasz kod DRF (widoki, serializery, routing) i automatycznie wygenerować dokumentację. My użyjemy popularnej i nowoczesnej biblioteki `drf-spectacular`.

Krok 1: Instalacja

Najpierw musimy zainstalować bibliotekę w naszym wirtualnym środowisku.

```
pip install drf-spectacular
```

Krok 2: Konfiguracja w `settings.py`

Następnie dodajemy `drf_spectacular` do `INSTALLED_APPS` i konfigurujemy DRF, aby używał jej do generowania schematu API.

```
# myproject/settings.py

INSTALLED_APPS = [
    # ... inne aplikacje
    'rest_framework',
    'drf_spectacular', # Dodajemy bibliotekę
    # ...
]

# Dodajemy konfigurację dla DRF
REST_FRAMEWORK = {
    # Ustawiamy drf-spectacular jako domyślny generator schematu
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
    # ... inne ustawienia DRF, np. paginacja, uwierzytelnianie
}

# Opcjonalna, ale zalecana konfiguracja dla drf-spectacular
SPECTACULAR_SETTINGS = {
    'TITLE': 'Moje Wspaniałe API Projektu',
    'DESCRIPTION': 'Dokumentacja dla API, które robi niesamowite rzeczy.',
    'VERSION': '1.0.0',
    'SERVE_INCLUDE_SCHEMA': False, # Zazwyczaj nie chcemy udostępniać pliku
    # schematu publicznie
}
```

Krok 3: Dodanie URLi do dokumentacji

Na koniec, musimy dodać ścieżki URL w głównym pliku `urls.py` naszego projektu, aby dokumentacja była dostępna w przeglądarce.

```
# myproject/urls.py
from django.contrib import admin
from django.urls import path, include
# Importujemy widoki z drf-spectacular
from drf_spectacular.views import SpectacularAPIView,
SpectacularSwaggerView, SpectacularRedocView

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/v1/', include('myapp.urls')), # Ścieżki do naszego API

    # Ścieżki do dokumentacji
    # Zwraca plik schematu .yaml
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    # Widok Swagger UI (rekomendowany)
```

```

    path('api/schema/swagger-ui/',
SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-ui'),
    # Alternatywny widok ReDoc
    path('api/schema/redoc/',
SpectacularRedocView.as_view(url_name='schema'), name='redoc'),
]

```

Po uruchomieniu serwera (`python manage.py runserver`), wejdź na adres `http://127.0.0.1:8000/api/schema/swagger-ui/` . Powinieneś zobaczyć pięknie wygenerowaną, interaktywną dokumentację swojego API!

4. Ulepszanie automatycznie generowanej dokumentacji

`drf-spectacular` świetnie radzi sobie z analizą serializatorów i widoków, ale możemy mu pomóc, dodając więcej szczegółów za pomocą docstringów i specjalnych dekoratorów.

Użycie docstringów

Biblioteka automatycznie zaciąga komentarze (docstringi) z klas widoków (np. `APIView` , `ViewSet`) i używa ich jako opisu dla grupy endpointów.

```

# myapp/views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from .serializers import ProductSerializer
from .models import Product

class ProductListView(APIView):
    """
    Ten widok obsługuje operacje na liście produktów.

    * Pozwala na pobranie listy wszystkich produktów.
    * Pozwala na dodanie nowego produktu.
    """
    def get(self, request):
        # ... logika pobierania produktów
        products = Product.objects.all()
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)

    def post(self, request):
        # ... logika tworzenia produktu
        pass

```

Dekorator `@extend_schema`

To najpotężniejsze narzędzie do dostosowywania dokumentacji. Pozwala na modyfikację niemal każdego aspektu opisu pojedynczego endpointu.

Tip

Dekorator `@extend_schema` importujemy z `drf_spectacular.utils`. Możemy go używać nad metodami w widokach opartych na klasach (`get`, `post` itd.) lub nad widokami opartymi na funkcjach.

Przykład 1: Dodawanie podsumowania, opisu i tagów

```
from drf_spectacular.utils import extend_schema

class ProductDetailView(APIView):
    @extend_schema(
        summary="Pobierz szczegóły jednego produktu",
        description="Zwraca pełne informacje o produkcie na podstawie jego ID.",
        tags=['Produkty'] # Grupuje endpointy w UI
    )
    def get(self, request, pk):
        # ... logika
        pass
```

Przykład 2: Opisanie parametrów zapytania (query params)

```
from drf_spectacular.utils import extend_schema, OpenApiParameter
from drf_spectacular.types import OpenApiTypes

class ProductListView(APIView):
    @extend_schema(
        parameters=[
            OpenApiParameter(
                name='category',
                description='Filtruj produkty po ID kategorii',
                required=False,
                type=OpenApiTypes.INT
            ),
        ],
        tags=['Produkty']
    )
    def get(self, request):
        # Możemy teraz pobrać parametr:
```

```
category_id = request.query_params.get('category')
# ... logika filtrowania
pass
```

Przykład 3: Opisanie różnych kodów odpowiedzi

```
from drf_spectacular.utils import extend_schema, OpenApiResponse
from .serializers import ProductSerializer, ErrorSerializer # Załóżmy, że
mamy ErrorSerializer

class ProductDetailView(APIView):
    @extend_schema(
        summary="Usuń produkt",
        tags=['Produkty'],
        responses={
            204: OpenApiResponse(description="Produkt został pomyślnie
usunięty (brak zawartości)" ),
            404: OpenApiResponse(description="Nie znaleziono produktu o
podanym ID", response=ErrorSerializer),
        }
    )
    def delete(self, request, pk):
        # ... logika usuwania
        pass
```

5. Wzmianka o AI w kontekście API

Wasze API nie musi ograniczać się tylko do operacji CRUD na bazie danych. Może ono stanowić bramę do znacznie bardziej zaawansowanych funkcji, w tym modeli sztucznej inteligencji.

Note

Wyobraźmy sobie API, które oferuje usługę analizy sentymentu (wydźwięku) tekstu. Klient wysyła fragment tekstu, a serwer, używając biblioteki AI (np. transformers od Hugging Face lub API od OpenAI), analizuje go i zwraca wynik: "pozytywny", "negatywny" lub "neutralny".

W takim przypadku, świetna dokumentacja jest absolutnie kluczowa. Musi ona precyzyjnie opisywać:

- **Format wejściowy:** np. `{"text": "Kocham programować w Pythonie!"}`

- **Format wyjściowy:** np. `{"input_text": "...", "sentiment": "positive", "confidence_score": 0.98}`
- **Ograniczenia:** np. maksymalna długość tekstu.
- **Możliwe błędy:** co się stanie, jeśli tekst jest pusty?

Dzięki `drf-spectacular` możemy to wszystko elegancko udokumentować, sprawiając, że nasze "inteligentne" API będzie łatwe w użyciu dla każdego.

Django - Wielki Finał: Wybierz Swój Projekt!

Witajcie! Przeszliście długą drogę – od podstaw Pythona, przez bazy danych, Git, aż po zaawansowane funkcje frameworka Django. Zdobyliście solidną wiedzę, która teraz czeka na praktyczne zastosowanie.

Przed Wami zadanie końcowe: **stworzenie w pełni funkcjonalnej aplikacji webowej**. To szansa, aby połączyć wszystkie nabyte umiejętności w jednym, kompleksowym projekcie. Poniżej znajdziecie cztery propozycje. Każda z nich jest ciekawa i pozwoli Wam zmierzyć się z realnymi wyzwaniami programistycznymi.

Podczas zajęć wspólnie z prowadzącym zrealizujemy jeden z tych projektów krok po kroku, co będzie dla Was świetnym wsparciem i praktycznym pokazem dobrych praktyk.

Wybierzcie projekt, który najbardziej Was inspirowa i do dzieła!

✓ Wspólne Wymagania dla Wszystkich Projektów

Niezależnie od wybranego tematu, każda aplikacja musi spełniać poniższe kryteria. To fundament, na którym zbudujecie unikalne funkcjonalności.

- **Rejestracja i uwierzytelnianie:** Wykorzystaj wbudowany w Django system `django.contrib.auth`.
- **Rozbudowany panel admina:**
 - Dodaj pola wyszukiwania (`search_fields`) i filtry (`list_filter`).
 - Użyj `inlines` do edycji powiązanych modeli w jednym widoku.
 - Zadbaj o czytelne wyświetlanie danych (`list_display` z własnymi metodami).
- **Generowanie danych testowych:** Użyj bibliotek **Seeder/Faker**, aby szybko wypełnić bazę danych realistycznymi danymi.
- **Komenda `management`:** Stwórz własną komendę (np. `python manage.py seed_db`), która uruchomi proces generowania danych.
- **Testy jednostkowe:** Napisz kilka podstawowych testów, aby sprawdzić kluczowe funkcjonalności Twojej aplikacji (np. czy modele poprawnie się tworzą, czy widoki zwracają kod 200).

- **Obsługa mediów:** Wykorzystaj `ImageField` do przesyłania i wyświetlania obrazków (np. okładek, zdjęć aktorów, plakatów).
- **Estetyczny interfejs:** Aplikacja powinna być przyjazna dla użytkownika i odporna na podstawowe błędy (np. próba dostępu do zastrzeżonej strony przez niezalogowanego użytkownika).

Projekty do Wyboru

Poniżej znajdują się cztery propozycje. Każda z nich kładzie nacisk na nieco inne aspekty pracy z Django. Zastanów się, co chciałbyś przećwiczyć najlepiej!

1. Strona Kina - "KinoMania"

Stwórz serwis internetowy dla kina, w którym użytkownicy mogą przeglądać repertuar, poznawać szczegóły filmów i rezerwować bilety na seanse. To świetny projekt do przećwiczenia złożonych relacji między modelami i logiki biznesowej.

Główne Modele

```
erDiagram
    Film {
        string tytul
        text opis
        date data_premiery
        ImageField plakat
    }
    Aktor {
        string imie_nazwisko
        ImageField zdjecie
    }
    Rezyser {
        string imie_nazwisko
        ImageField zdjecie
    }
    Gatunek {
        string nazwa
    }
    Seans {
        datetime czas_rozpoczecia
        decimal cena
    }
    Rezerwacja {
        int ilosc_miejsc
        string status
    }
```

```

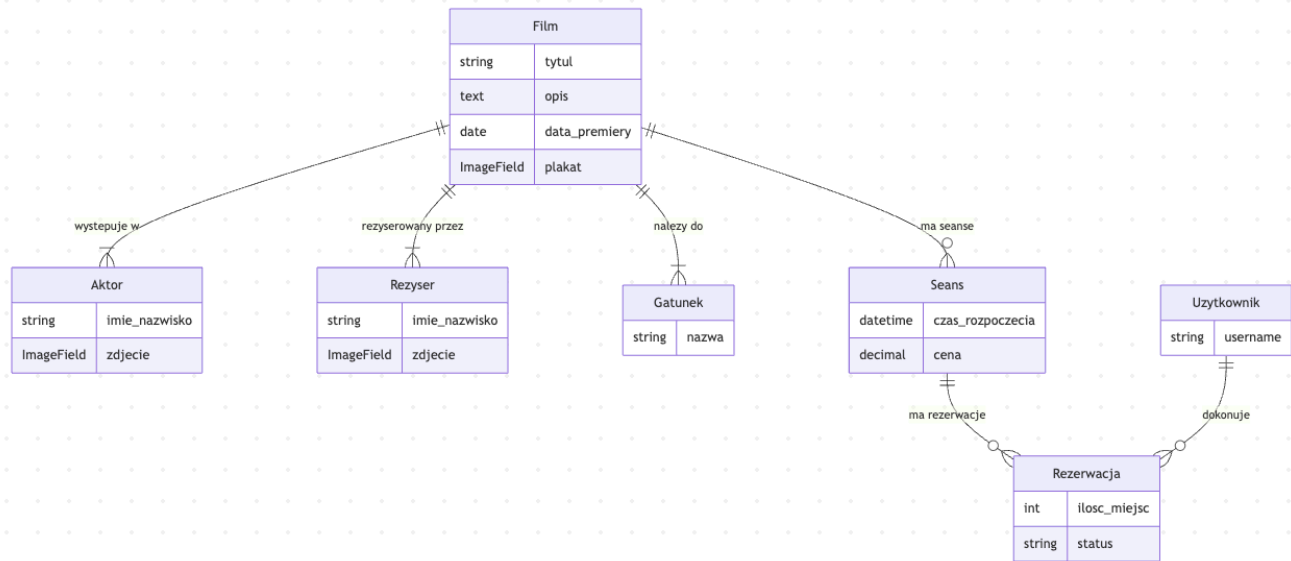
}
Uzytkownik {
    string username
}

```

```

Film ||--|{ Gatunek : "należy do"
Film ||--|{ Aktor : "występuje w"
Film ||--|{ Rezyser : "reżyserowany przez"
Film ||--o{ Seans : "ma seanse"
Seans ||--o{ Rezerwacja : "ma rezerwacje"
Uzytkownik ||--o{ Rezerwacja : "dokonuje"

```



Kluczowe Funkcjonalności

- Przeglądanie listy filmów w repertuarze.
- Strona szczegółów filmu z obsadą, reżyserem, opisem i zwiastunem.
- Wyświetlanie harmonogramu seansów na dany dzień.
- System rezerwacji biletów na wybrany seans (tylko dla zalogowanych).
- Panel użytkownika z listą jego rezerwacji.

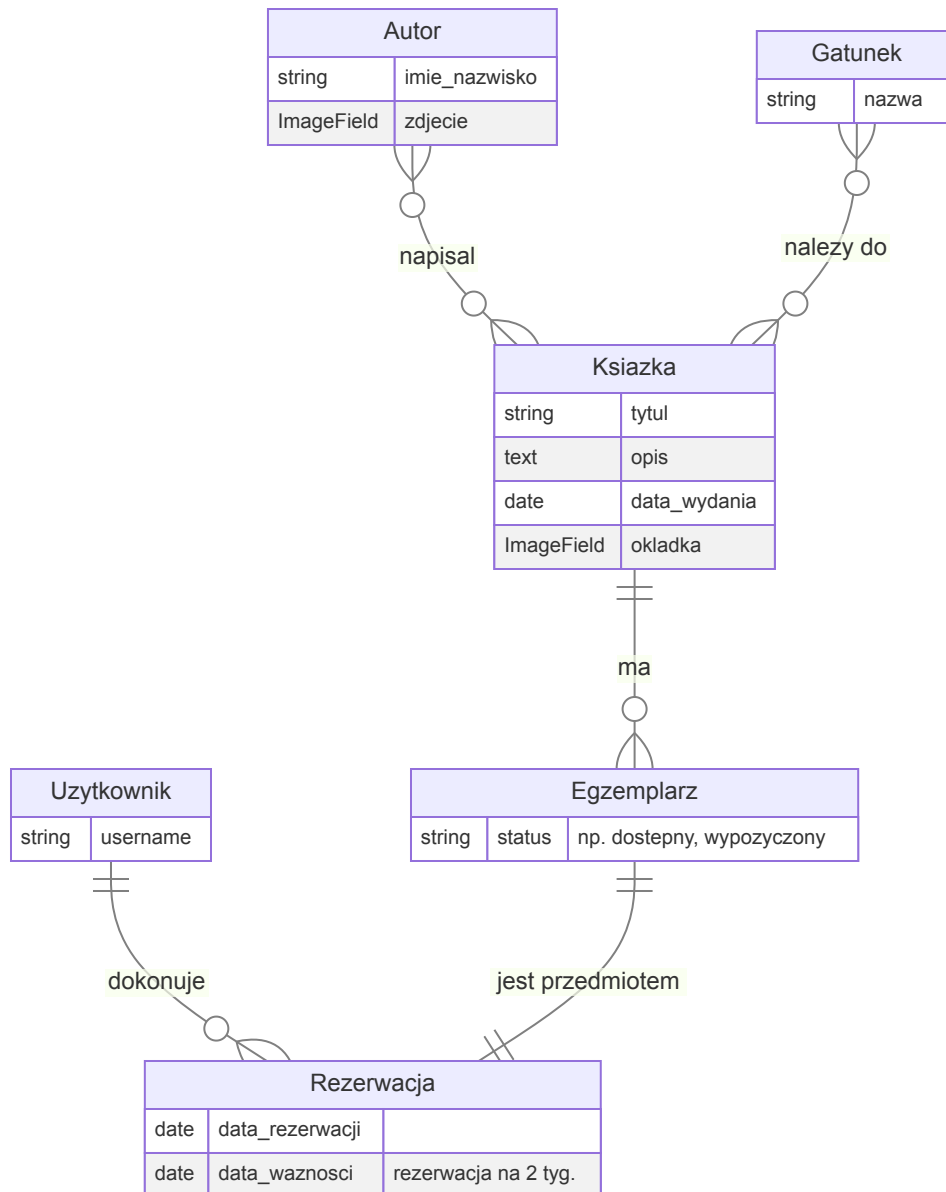
Co przećwiczysz?

- Złożone relacje **ManyToMany** (Film-Aktor, Film-Gatunek).
- Pracę z datą i czasem przy obsłudze seansów.
- Logikę biznesową (np. sprawdzanie dostępności miejsc).
- Zaawansowane zapytania **QuerySet** do filtrowania repertuaru.

2. Strona Biblioteki - "BiblioTech"

Zaprojektuj i wdróż system do zarządzania katalogiem książek w bibliotece. Użytkownicy mogą przeglądać zbiory, wyszukiwać pozycje i rezerwować je online. To doskonała okazja do pracy nad logiką stanów i zarządzaniem zasobami.

Główne Modele



Kluczowe Funkcjonalności

- Katalog książek z możliwością filtrowania po autorze, gatunku.
- Wyszukiwarka tekstowa.
- Strona szczegółów książki z informacją o dostępnych egzemplarzach.
- Możliwość rezerwacji dostępnego egzemplarza na 2 tygodnie (tylko dla zalogowanych).

- Panel użytkownika z listą aktualnych i historycznych rezerwacji.

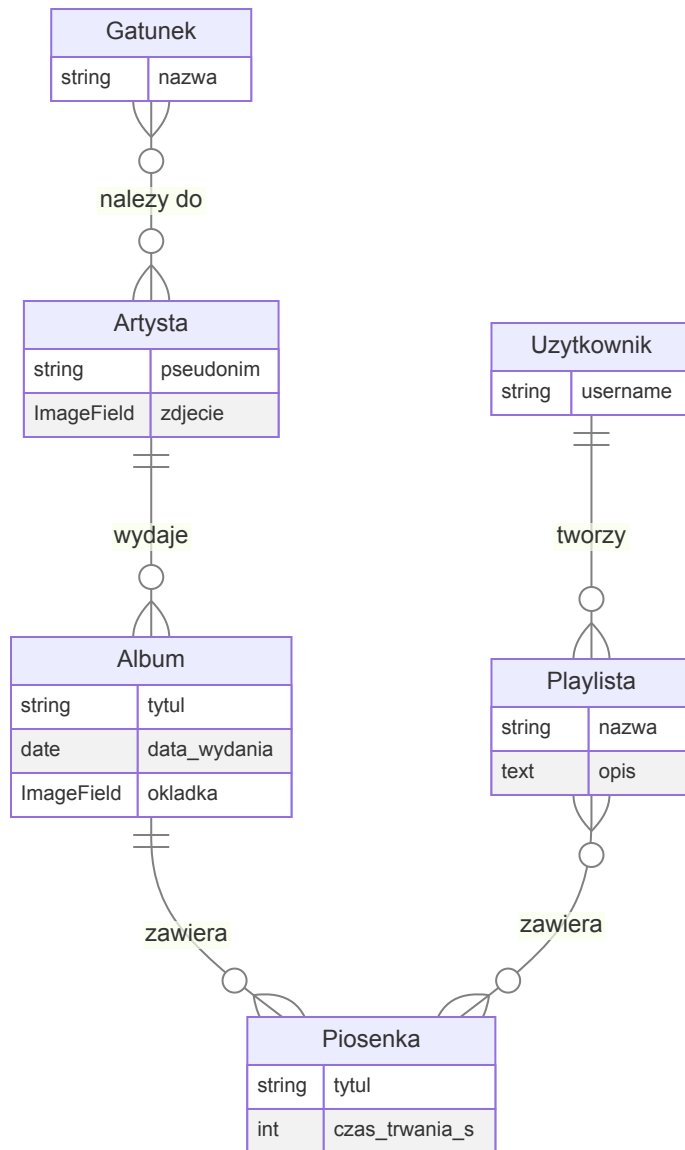
Co przećwiczysz?

- Relację `OneToMany` (jedna `Książka` może mieć wiele `Egzemplarzy`).
- Zarządzanie stanami (dostępność egzemplarza).
- Pracę z modulem `datetime` do obsługi terminów rezerwacji.
- Transakcje bazodanowe, aby uniknąć rezerwacji tego samego egzemplarza przez dwie osoby naraz.

3. Portal Muzyczny - "BeatHub"

Stwórz platformę, która pozwoli użytkownikom odkrywać nową muzykę. Aplikacja będzie agregować informacje o artystach, ich albumach i piosenkach, a zalogowani użytkownicy będą mogli tworzyć własne, unikalne playlisty.

Główne Modele



Kluczowe Funkcjonalności

- Baza artystów, albumów i piosenek.
- Strona artysty z listą jego albumów.
- Strona albumu z listą piosenek.
- Możliwość tworzenia, edycji i usuwania własnych playlist (tylko dla zalogowanych).
- Dodawanie i usuwanie piosenek z playlist.
- Publiczne profile użytkowników z ich playlistami.

Co przećwiczysz?

- Bardzo intensywną pracę z relacjami **ManyToMany** (Playlist-Piosenka).
- Możliwość wykorzystania niestandardowej tabeli pośredniej (**through**) do zapisania np. kolejności piosenek na playliście.

- Zagnieżdżone `QuerySet` (np. znajdź wszystkie piosenki artysty X z gatunku Y).
- Formularze i `FormSets` do zarządzania playlistami.

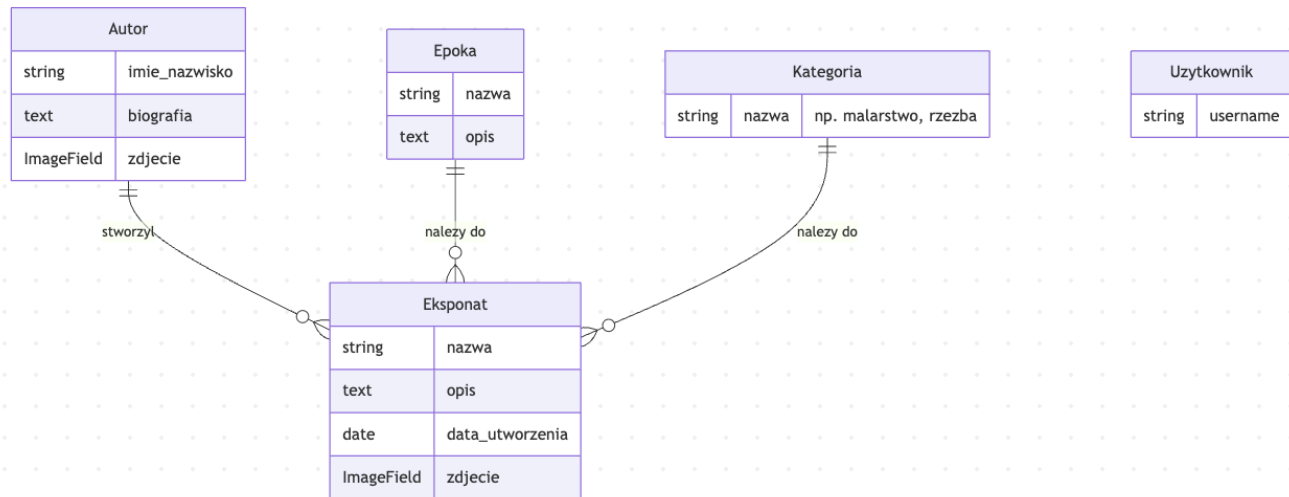
4. Wirtualne Muzeum - "ArtExplorer"

Zbuduj stronę internetową muzeum, która pozwoli zwiedzającym na wirtualne odkrywanie zbiorów. Użytkownicy będą mogli przeglądać eksponaty, poznawać ich historię i autorów, a także grupować je według epok czy kategorii.

Główne Modele

```
erDiagram
    Autor {
        string imie_nazwisko
        text biografia
        ImageField zdjecie
    }
    Epoka {
        string nazwa
        text opis
    }
    Kategoria {
        string nazwa "np. malarstwo, rzeźba"
    }
    Eksponat {
        string nazwa
        text opis
        date data_utworzenia
        ImageField zdjecie
    }
    Uzytkownik {
        string username
    }

    Autor ||--o{ Eksponat : "stworzył"
    Epoka ||--o{ Eksponat : "należy do"
    Kategoria ||--o{ Eksponat : "należy do"
```



Kluczowe Funkcjonalności

- Galeria eksponatów z zaawansowanym filtrowaniem (po autorze, epoce, kategorii).
- Strona szczegółów eksponatu z wysokiej jakości zdjęciem i pełnym opisem.
- Strony biograficzne autorów z listą ich dzieł w muzeum.
- Strony tematyczne dla epok i kategorii.
- Możliwość stworzenia "wirtualnej wycieczki" (np. jako prosta lista ulubionych eksponatów dla zalogowanego użytkownika).

Co przećwiczysz?

- Budowanie systemów kategoryzacji i tagowania.
- Intensywną pracę z `ImageField` i potencjalnie z bibliotekami do obróbki obrazów (np. `Pillow`).
- Tworzenie przejrzystych i informacyjnych widoków opartych na filtrowaniu `QuerySet`.
- Projektowanie modeli z myślą o dużej ilości treści tekstowej i graficznej.

Jak Zacząć? Krótki Plan Działania

1. **Wybierz projekt**, który najbardziej Cię interesuje.
2. **Zaprojektuj modele**. Zastanów się nad polami i relacjami. Narysuj schemat (możesz użyć Mermaid w Obsidianie!).
3. **Stwórz projekt i aplikację Django**.
4. **Zdefiniuj modele** w pliku `models.py`.
5. **Uruchom migracje** (`makemigrations`, `migrate`).
6. **Skonfiguruj Panel Admina**, aby wygodnie zarządzać danymi.

7. **Napisz seeder**, aby wypełnić bazę danymi testowymi.
8. Zaczynij tworzyć **widoki i szablony**, implementując kolejne funkcjonalności.
9. **Pisz testy** na bieżąco!

Powodzenia!